multiple_linear_regression

June 8, 2026

0.0.1 Multiple Linear Regression

[1]:
```
# import required libraries
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
%matplotlib inline
```

[2]:
```
# read csv data and load into variable
df = pd.read_csv('economic_index.csv')
df.head()
```

[2]:

	Unnamed: 0	year	month	interest_rate	unemployment_rate	index_price
0	0	2017	12	2.75	5.3	1464
1	1	2017	11	2.50	5.3	1394
2	2	2017	10	2.50	5.3	1357
3	3	2017	9	2.50	5.3	1293
4	4	2017	8	2.50	5.4	1256

[3]:
```
# drop unncessary columns
df.drop(columns=["Unnamed: 0", "year", "month"], axis=1, inplace=True)
```

[4]:
```
df.head()
```

[4]:

	interest_rate	unemployment_rate	index_price
0	2.75	5.3	1464
1	2.50	5.3	1394
2	2.50	5.3	1357
3	2.50	5.3	1293
4	2.50	5.4	1256

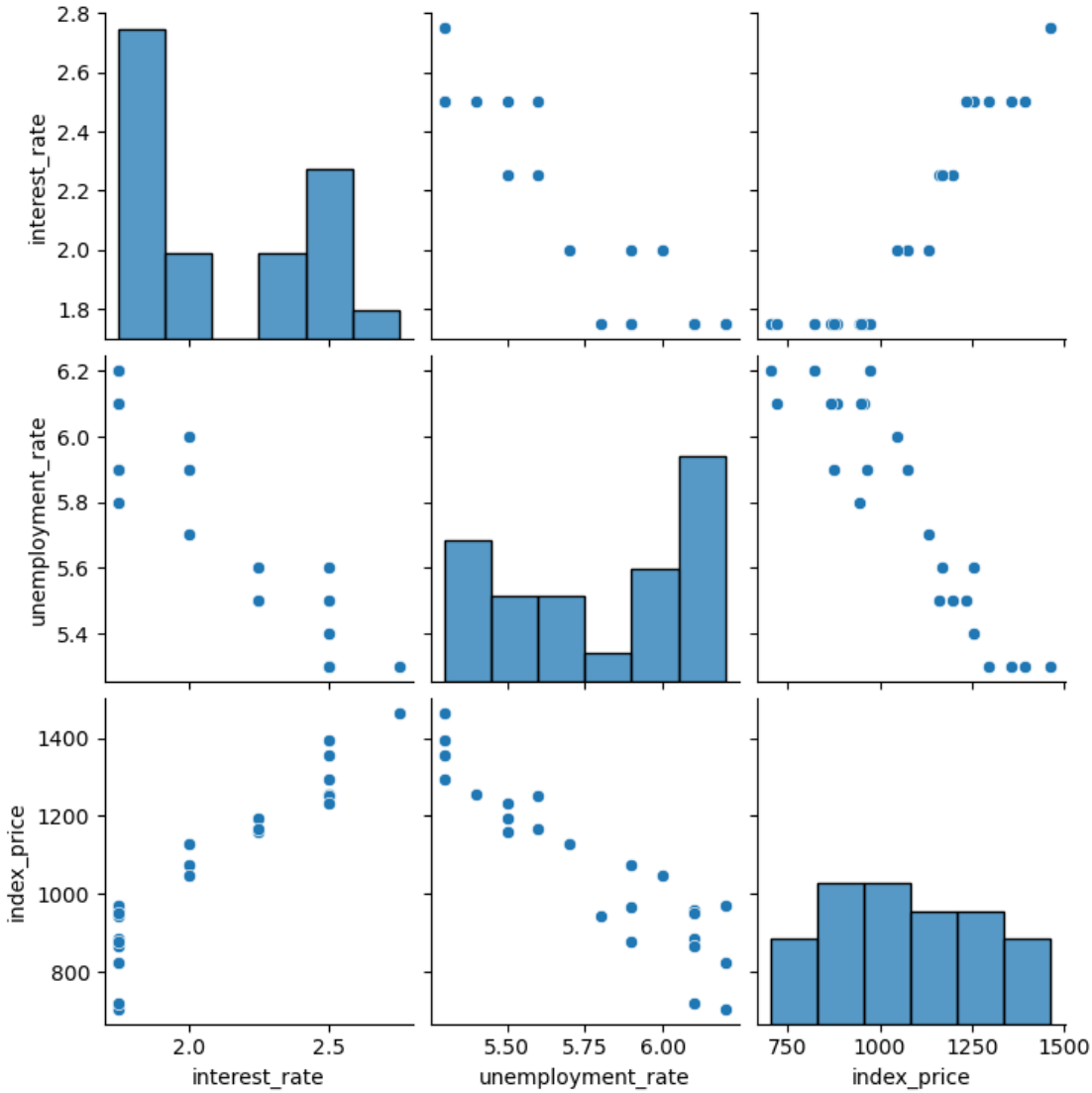
[5]:
```
df.isnull().sum()
```

[5]:
```
interest_rate        0
unemployment_rate    0
index_price          0
dtype: int64
```

Visualization

[6]:
```
import seaborn as sns
sns.pairplot(df)
```

[6]:
```
<seaborn.axisgrid.PairGrid at 0x78facaa0db20>
```



[7]:
```
df.corr()
```

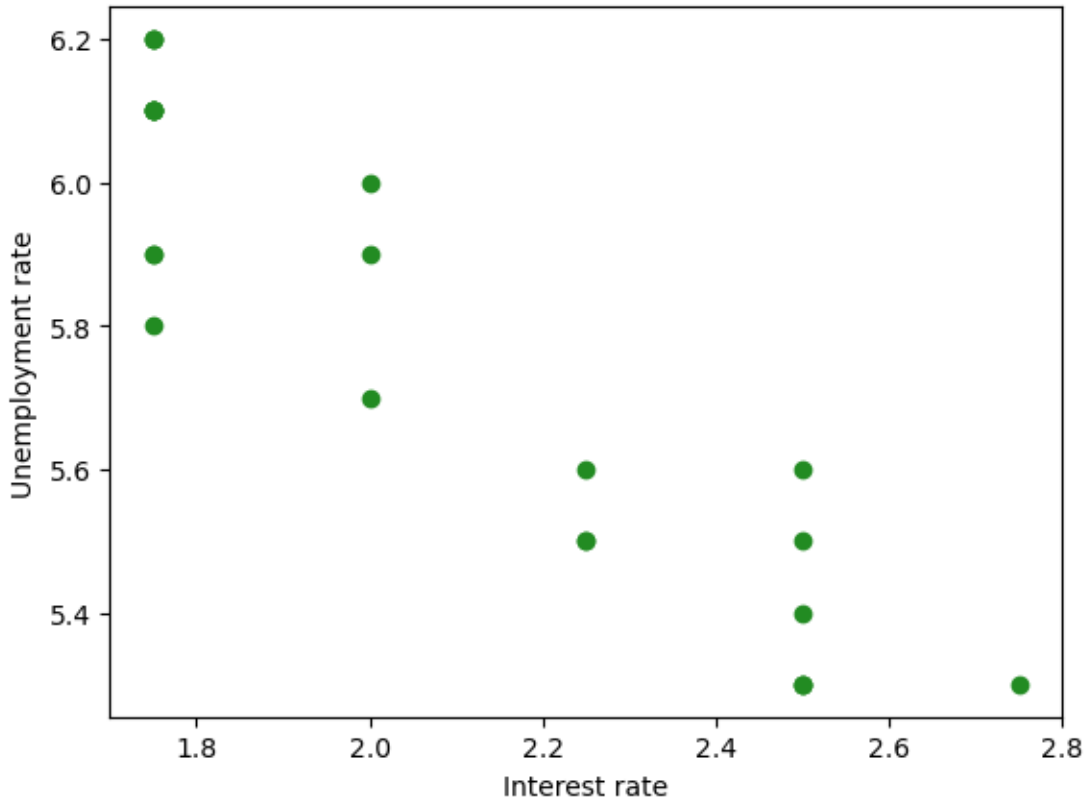
[7]:

	interest_rate	unemployment_rate	index_price
interest_rate	1.000000	-0.925814	0.935793
unemployment_rate	-0.925814	1.000000	-0.922338
index_price	0.935793	-0.922338	1.000000

Visualizing data points

[8]:
```
plt.scatter(df['interest_rate'], df['unemployment_rate'], c='forestgreen')
plt.xlabel('Interest rate')
plt.ylabel('Unemployment rate')
```

[8]:
```
Text(0, 0.5, 'Unemployment rate')
```



[9]:
```
# Independent and Dependent features
X = df.iloc[:,:-1]
y = df.iloc[:,-1]
```

[10]:
```
X.head()
```

[10]:

	interest_rate	unemployment_rate
0	2.75	5.3
1	2.50	5.3
2	2.50	5.3
3	2.50	5.3
4	2.50	5.4

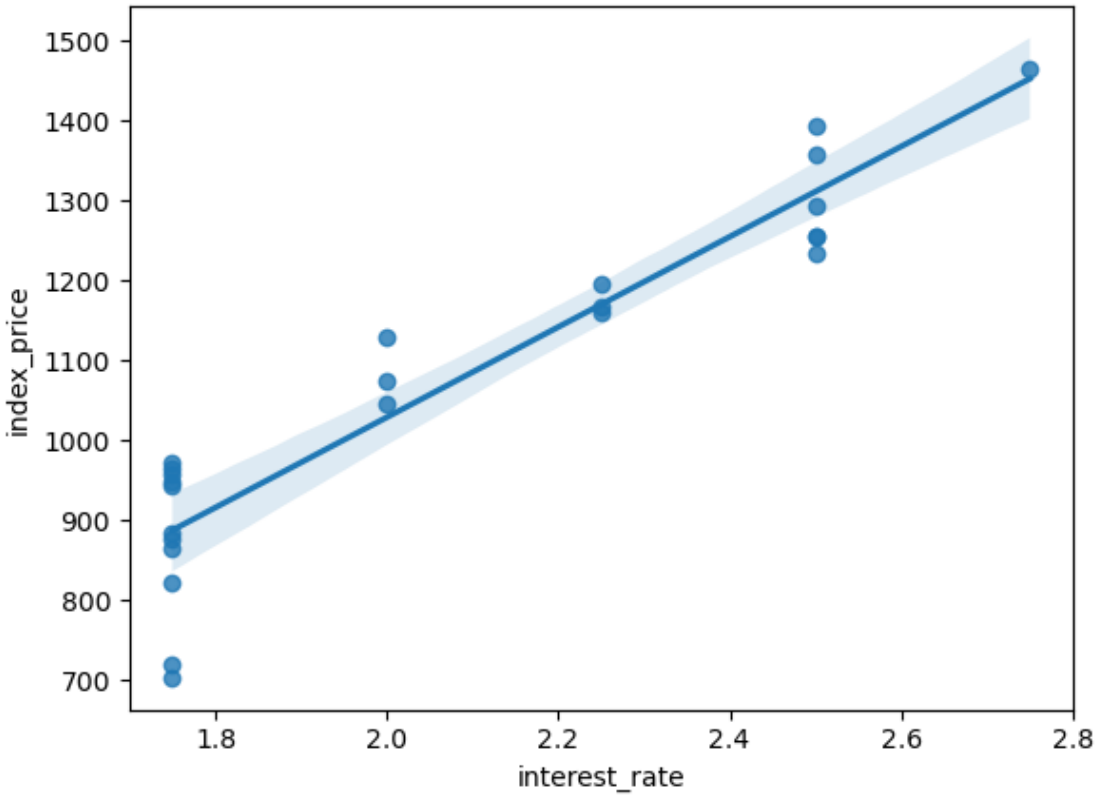
[11]:
```
# Train test split
from sklearn.model_selection import train_test_split
```

[12]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,␣
↪random_state=42)
```

[13]:
```
import seaborn as sns
```

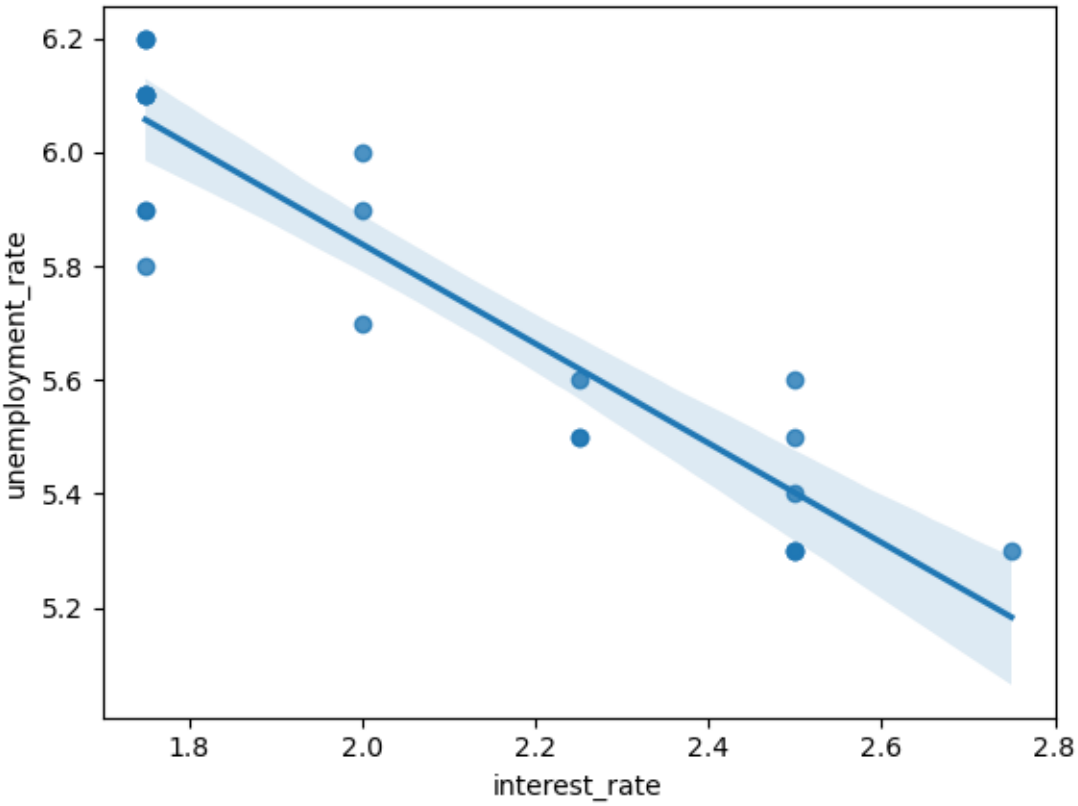
[14]:
```
sns.regplot(x= df['interest_rate'], y = df['index_price'])
```

[14]: <Axes: xlabel='interest_rate', ylabel='index_price'>



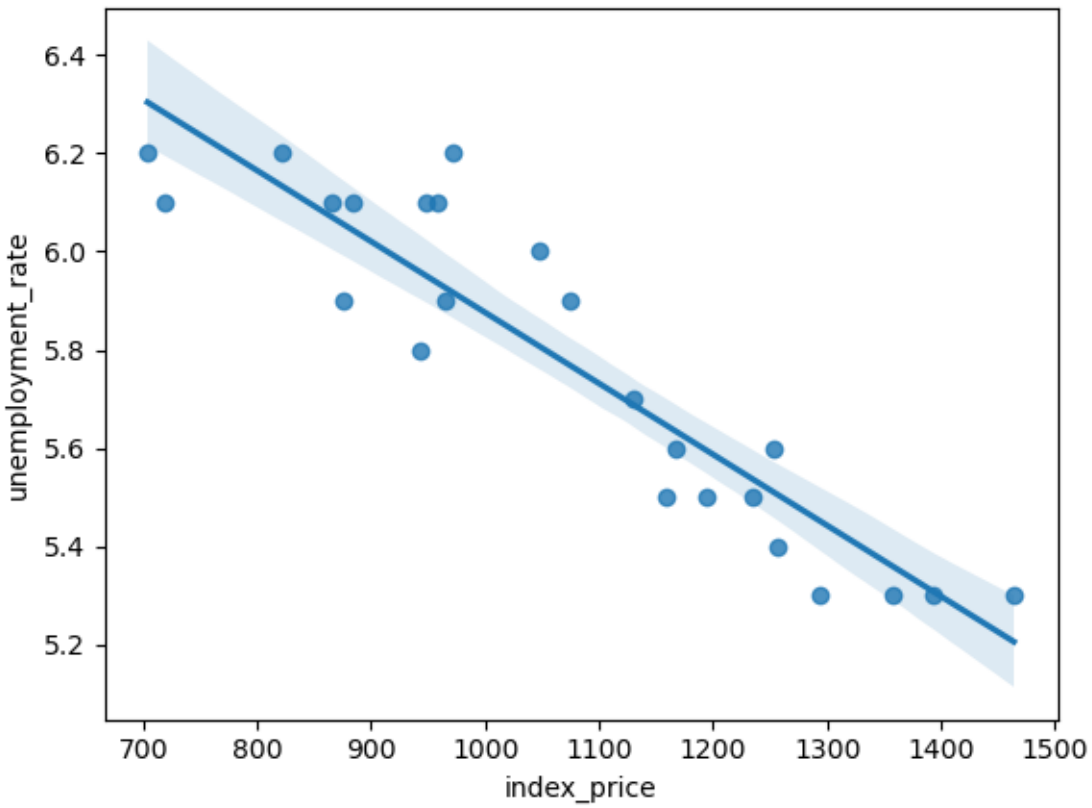
[15]:
```
sns.regplot(x= df['interest_rate'], y = df['unemployment_rate'])
```

[15]: <Axes: xlabel='interest_rate', ylabel='unemployment_rate'>



[16]:
```
sns.regplot(x= df['index_price'], y = df['unemployment_rate'])
```

[16]:
```
<Axes: xlabel='index_price', ylabel='unemployment_rate'>
```



[17]:
```
from sklearn.preprocessing import StandardScaler
```

[18]:
```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.fit_transform(X_test)
```

[19]:
```
X_train
```

[19]:
```
array([[-0.90115511,  0.37908503],
       [ 1.31077107, -1.48187786],
       [-0.90115511,  1.30956648],
       [ 1.31077107, -0.55139641],
       [ 1.31077107, -1.48187786],
       [-0.16384638,  0.68924552],
       [-0.90115511,  0.999406  ],
       [ 1.31077107, -1.48187786],
       [ 1.31077107, -1.17171738],
       [-0.90115511,  1.30956648],
       [-0.90115511,  0.999406  ],
       [-0.90115511,  0.37908503],
       [-0.90115511,  0.999406  ],
```

```
       [ 0.57346234, -0.8615569 ],
       [-0.16384638, -0.24123593],
       [-0.90115511,  0.06892455],
       [-0.90115511,  0.999406  ],
       [ 1.31077107, -0.8615569 ]])
```

[20]:
```
from sklearn.linear_model import LinearRegression
regression = LinearRegression()
```

[21]:
```
regression.fit(X_train, y_train)
```

[21]:
```
LinearRegression()
```

[22]:
```
# Cross validation
from sklearn.model_selection import cross_val_score

validation_score = cross_val_score(regression, X_train, y_train,␣
 ↪scoring='neg_mean_squared_error', cv=3)
```

[23]:
```
np.mean(validation_score)
```

[23]:
```
-5914.828180162396
```

[24]:
```
# Prediction
y_pred = regression.predict(X_test)

y_pred
```

[24]:
```
array([1180.7466813 ,  802.74279699, 1379.83457045,  838.52599602,
       973.85313963, 1144.96348227])
```

[25]:
```
# Performance metrics
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

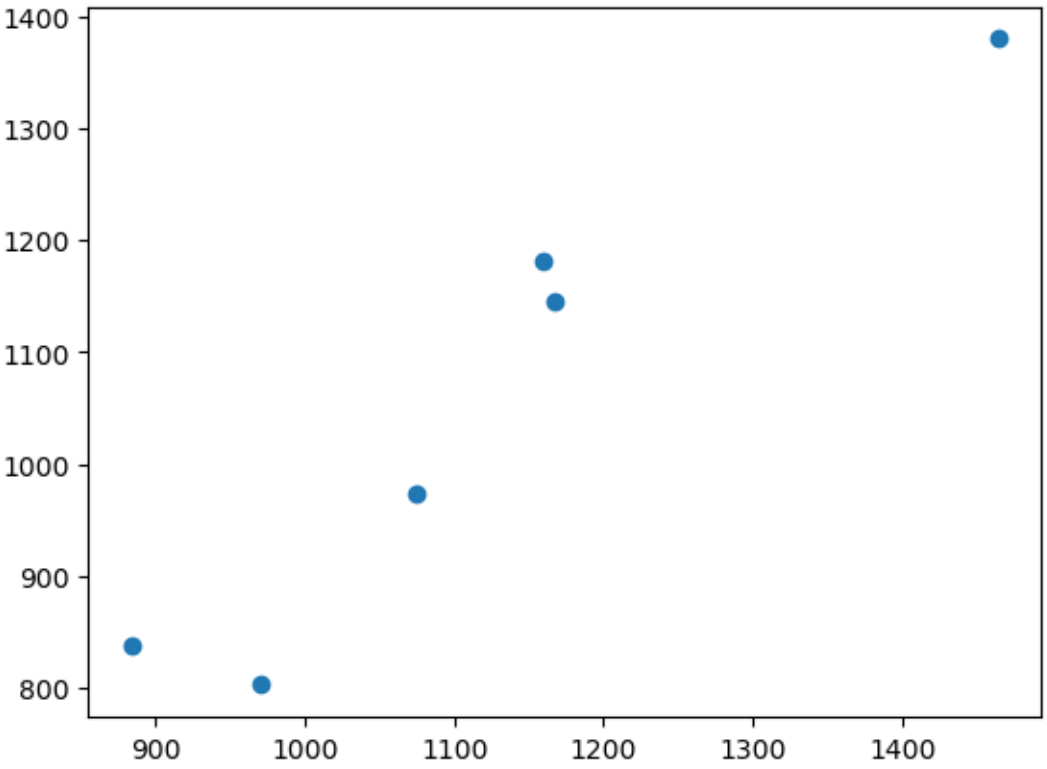
[26]:
```
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mse)

print(mse)
print(mae)
print(rmse)
```

```
8108.567426306611
73.80444932337099
90.04758423359624
```

[27]:
```
plt.scatter(y_test, y_pred)
```

[27]: <matplotlib.collections.PathCollection at 0x78fa9c8725a0>

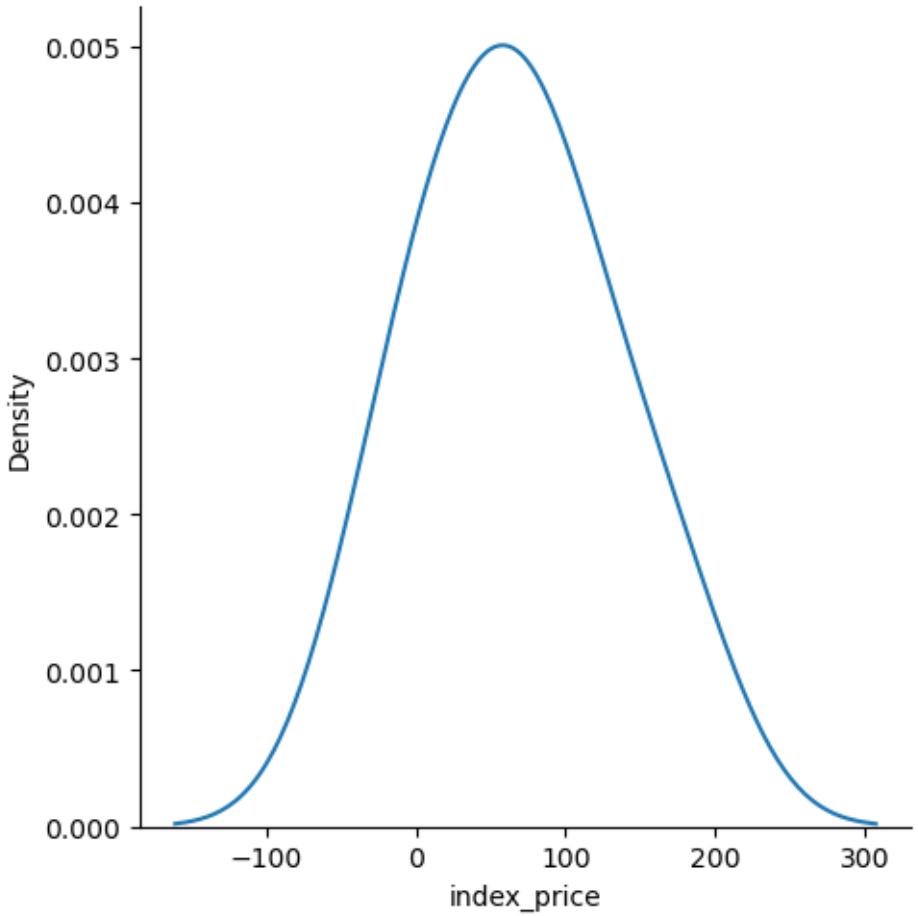


[28]:
```
residuals = y_test - y_pred
print(residuals)
```

```
8     -21.746681
16    168.257203
0      84.165430
18     45.474004
11    101.146860
9      22.036518
Name: index_price, dtype: float64
```

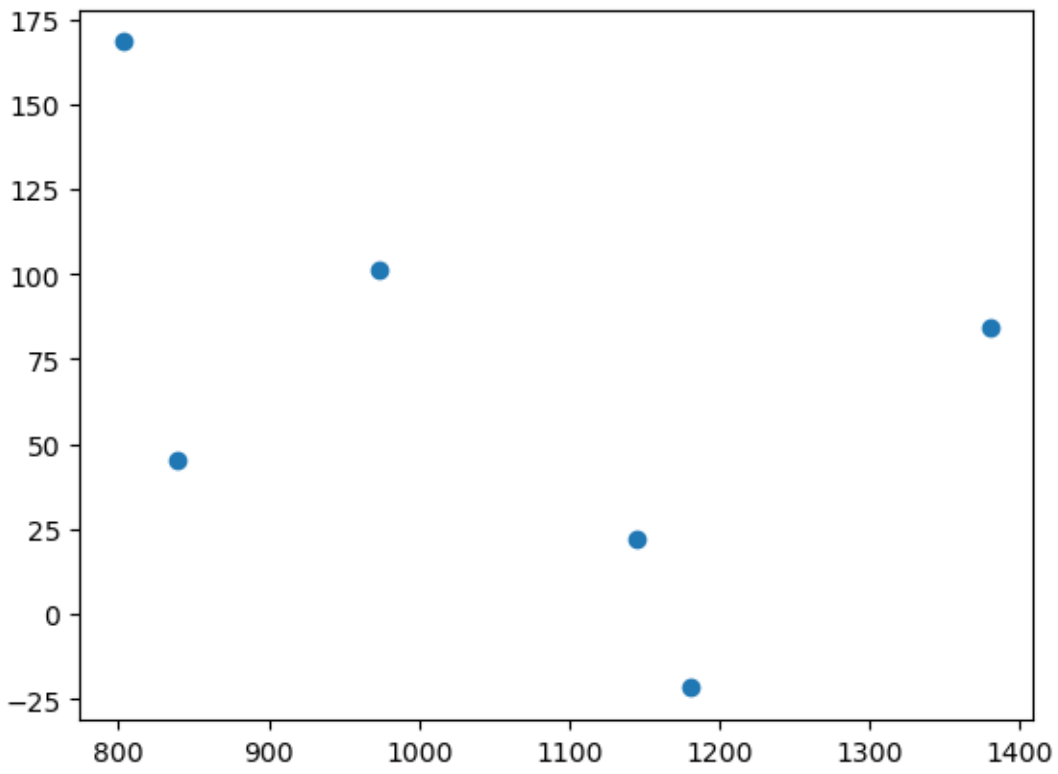
[29]:
```
# plot residuals
sns.displot(residuals, kind='kde')
```

[29]: <seaborn.axisgrid.FacetGrid at 0x78fa9c73cce0>



[30]:
```
# Scatter plot with respect to prediction and residuals
plt.scatter(y_pred, residuals)
```

[30]: <matplotlib.collections.PathCollection at 0x78fa9c212000>


175
150
125
100
75
50
25
0
−25
800
900
1000
1100
1200
1300
1400

[ ]: